

urdu-slm: Brief

A from-scratch Urdu language model, summarized

Explainer generated from the project source and README

Scope and honesty statement

This is a concise, accurate summary of the **urdu-slm** repository as it exists today. The project is explicitly **phase 1**: a complete data pipeline, tokenizer, and model architecture, plus a tiny 13M-parameter model trained on a CPU as a proof of concept. **Phase 2** — training the real ~124M-parameter **base** model on a GPU — has *not* happened. No base-model weights or metrics exist; every number below belongs to the tiny CPU proof run unless stated otherwise. For full technical depth see **deep-dive.pdf**.

1 What it is

urdu-slm builds a GPT-style language model for Urdu (a Perso-Arabic, right-to-left language) entirely from scratch: a custom-trained tokenizer and a hand-written PyTorch transformer, with no modeling code from the **transformers** library. The motivation, per the README: most hobby "train a GPT" projects reach for English data and a pretrained tokenizer, skipping every problem a low-resource, non-Latin script actually exposes — script normalization, a tokenizer that does not shred the language byte-by-byte, script-aware filtering, and an honest reckoning with a small data budget.

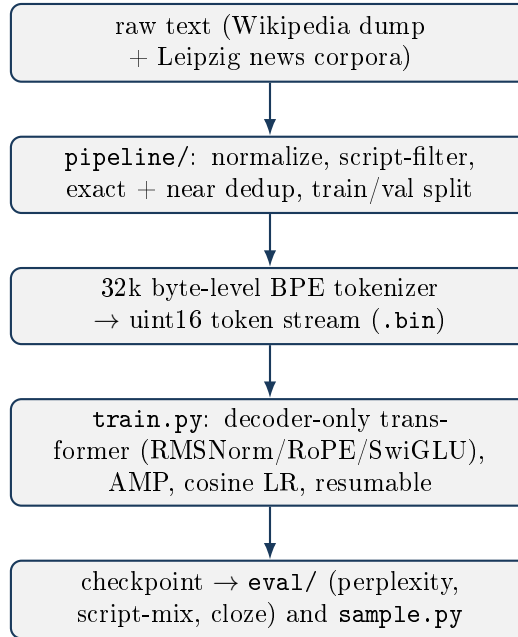


Figure 1: The full pipeline, one stage per line.

2 The data pipeline

Two no-authentication public sources: the Urdu Wikipedia dump (CC BY-SA 4.0) and two Wortschatz Leipzig newscrawl corpora (CC BY-NC 4.0, so any weights trained on them inherit a non-commercial restriction). Wikipedia’s wikitext markup is stripped with `mwparserfromhell`; Leipzig sentences are read directly from tab-separated text inside their `tar.gz` archives. Every stage streams (never loads the whole corpus into memory) and is *resumable*: a `.stage.done` marker lets a re-run skip already-finished stages.

Normalization: Unicode NFC composition, plus folding a handful of Arabic/Urdu encoding variants that are the same letter with different code points (e.g. Arabic Kaf $\rightarrow$ Urdu Keheh), while deliberately *keeping* short-vowel diacritics where present.

Filtering: a paragraph is dropped if it is under 40 characters or 6 words, if less than 65% of its letters fall in the Arabic Unicode script range (the project’s script-range heuristic for "is this Urdu", chosen instead of a heavyweight fastText language-ID model to stay dependency-light), or if under half its characters are alphabetic (catches leftover tables/markup).

Deduplication: exact duplicates via a blake2b hash set, and near duplicates via a vectorized MinHash + LSH (Locality-Sensitive Hashing) pass over character 5-gram shingles. The README documents rewriting the original per-shingle MinHash update loop (measured at ~ 250 docs/s) into two vectorized NumPy operations per document using `xxhash`, reaching ~ 2400 docs/s ($\sim 10\times$) with identical near-duplicate detection.

Measured corpus (from `data/corpus_stats.json`):

Source	Raw units	Kept after dedup	Chars
Urdu Wikipedia	1,728,010 articles	1,309,831	370.2M
Leipzig newscrawl 2016	1,000,000 sentences	930,452	122.7M
Leipzig newscrawl 2011	300,000 sentences	274,799	32.2M
Total	3,695,487	2,515,082 docs	438.5M

Filtering + dedup removed 31.9% of extracted paragraphs (mostly Urdu Wikipedia’s large population of near-identical bot-generated stub articles). Final split: 2,489,976 train docs / 25,106 val docs (deterministic 1% holdout), tokenizing to **111.7M** training tokens + 1.1M val tokens.

3 The tokenizer

A 32,000-token **byte-level Byte-Pair Encoding (BPE)** tokenizer, trained from scratch on the cleaned corpus. Byte-level means the starting alphabet is the 256 raw byte values, guaranteeing zero out-of-vocabulary symbols on any input. Measured against GPT-2’s tokenizer on an 806,495-character held-out Urdu sample: ours reaches 4.078 chars/token versus GPT-2’s 0.779 chars/token — GPT-2 needs **5.23x more tokens** to represent the same Urdu text, because its English-trained merges rarely match Urdu byte sequences.

4 The model

A decoder-only transformer (`model/transformer.py`), pre-norm, built block by block as:

$$x \rightarrow x + \text{Attention}(\text{RMSNorm}(x)) \rightarrow x + \text{SwiGLU}(\text{RMSNorm}(x))$$

Component glossary:

- **RMSNorm**: a cheaper LayerNorm variant that rescales a vector by its root-mean-square magnitude instead of centering and scaling it.
- **RoPE (rotary position embedding)**: encodes token position by rotating pairs of dimensions of the query/key vectors, so attention scores naturally depend on relative distance between tokens.
- **Causal self-attention**: each position attends only to itself and earlier positions, computed here with PyTorch’s fused `scaled_dot_product_attention(is_causal=True)` (Flash-attention kernels on supported GPUs), with grouped-query attention supported in the code (not enabled in any shipped preset).
- **SwiGLU**: a gated feed-forward layer (silu-activated gate $\times$ an up-projection) sized at an 8/3 hidden ratio to match a classic 4x-GELU MLP’s parameter count.
- **Tied embeddings**: the input token-embedding matrix and output vocabulary-prediction matrix are the literal same weights, halving that cost.

preset	params	layers	d_model	heads	context
tiny	13.0M	6	256	4	512
small	35.7M	6	512	8	1024
base	123.7M	14	768	12	1024

At a 32k vocabulary, the tied embedding table alone is 8.2M parameters, so even **tiny** is embedding-dominated; the README reports this split openly rather than shrinking the vocabulary to hit a rounder number.

5 Training and evaluation

`train.py` is configured by a target **token count** (`max_tokens` in YAML) rather than by raw step count, with gradient accumulation, bf16 mixed precision on GPU (fp32 on CPU), a cosine learning-rate schedule with linear warmup, gradient clipping, and full resumability (model + optimizer + LR schedule + data cursor all restored on `-resume`). The same script runs single-GPU or multi-GPU via `torchrun` (DistributedDataParallel), auto-detected from environment variables.

`eval/` provides: held-out **perplexity** (e^{loss} , lower is better); a **script-mix diagnostic** measuring what share of generated characters are Arabic-script versus Latin (checks the model stays in Urdu); and a 45-item hand-written **cloze** (fill-in-the-blank) benchmark scored by comparing the language-model loss of each candidate completion — built and calibrated for the not-yet-trained phase-2 model, so near-chance results on the tiny model are expected and reported honestly rather than hidden.

6 The tiny CPU proof run

Trained on an 8-core CPU purely to prove the whole pipeline works end to end: 500 steps, 4.10M tokens, train loss 9.74 $\rightarrow$ 5.42, held-out val loss 5.87 $\rightarrow$ 5.36 during training, final held-out perplexity 248.8, script-mix 98.2% Arabic-script output, cloze accuracy 0.378 (random baseline 0.333). The README is explicit that generated samples are "barely coherent by design" — real Urdu vocabulary, loosely on-topic, loose grammar — consistent with a 13M model at 500 steps, not a capability claim.

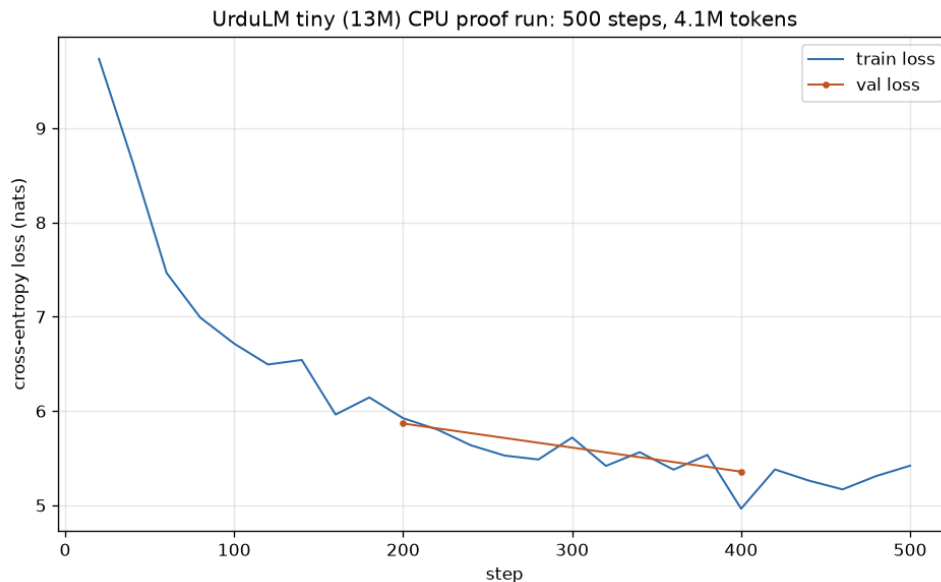


Figure 2: Train/val loss over the 500-step CPU proof run.

7 Testing

`pytest tests/` runs **12 tests**, all passing: 5 model tests (forward-pass shapes, a black-box test that perturbing a future token cannot change earlier logits, tied-embedding identity, generation smoke test, and per-preset parameter-count bands) and 7 pipeline tests (normalization, script-ratio filtering, exact and near dedup).

8 Status

Phase 1: done and verified. Real corpus built, tokenizer trained, all tests passing, tiny model trained on CPU with a demonstrably decreasing loss and a saved, reproducible checkpoint.

Phase 2 (base model): not trained yet, per the README — "a budget decision, not an engineering one." `configs/base.yaml` and `docs/GPU_RUN.md` document a ready-to-run launch, a Chinchilla-style token-budget analysis, and a cost estimate, but none of it has been executed.

The honest headline: data-limited, not compute-limited

Chinchilla-style scaling suggests ~ 20 training tokens per parameter, or $\sim 2.5\text{B}$ tokens for the 124M **base** model. The actual cleaned corpus is 111.7M tokens (~ 0.9 tokens/parameter, $\sim 4.5\%$ of that target). Rather than repeat the corpus ~ 22 times to hit the Chinchilla number (which would overfit), the documented plan trains 2–4 epochs ($\sim 220\text{M}$ – 450M tokens, what `configs/base.yaml` is set to) and treats growing the corpus, not spending more GPU time on the same data, as the real next lever. The estimated cost for the planned 450M-token single-A100 run is roughly \$6 (RunPod Community Cloud pricing, checked 2026-07-06) — an estimate, not a measurement; only the CPU proof run’s throughput has actually been measured.

9 Notable engineering decisions (from the README)

- Vectorizing the MinHash hot loop (two NumPy ops per document instead of a per-shingle Python loop) turned an unusably slow dedup stage into a fast one — profiling the real workload mattered more than picking a "fast" library.
- A "10M-parameter" model is largely fictional once you use a real 32k subword vocabulary; the embedding table dominates, so the project reports the embedding/block split honestly instead of shrinking the vocabulary to hit a headline number.
- RoPE plus `scaled_dot_product_attention(is_causal=True)` gives a correct, Flash-ready causal transformer with no hand-written mask, and the causal property is cheap to test directly by perturbing a future token.
- Planned next steps: parallelize the single-process Wikipedia extraction, persist MinHash signatures to disk for independent resumability, pre-filter Wikipedia bot stubs earlier, add an optional fastText language-ID fallback for mixed-script sources, and run tokenizer vocabulary-size ablations before committing the phase-2 vocabulary.

10 License

Code: MIT, Copyright (c) 2026 Salman Adnan. Urdu Wikipedia data is CC BY-SA 4.0; the Leipzig corpora are CC BY-NC 4.0, so released weights trained on them inherit a non-commercial restriction (`docs/DATA_LICENSES.md`).